

ヒープ

ソートアルゴリズムの回でも少し触れましたが、「**ヒープ (Heap)**」は二分木をベースにした非常に効率的なデータ構造です。

「ヒープ (積み上げた山)」という名前の通り、「**一番大きい (または小さい) 要素を常にすぐ取り出せるようにしておく**」という目的に特化しています。

1. ヒープの2つの絶対条件

ヒープとして成立するためには、以下の2つのルールを同時に満たす必要があります。

1. **構造の条件 (完全二分木)**：木に隙間がなく、上の階層から、そして左から順番にノードが埋まっていること。
 2. **値の条件 (親子関係)**：
 - **最大ヒープ**：親の値 子の値 (一番上が最大値)
 - **最小ヒープ**：親の値 子の値 (一番上が最小値)
-

2. なぜヒープを使うのか？ (メリット)

「データの山から一番大きな値を見つける」だけなら、ヒープの頂点を見るだけで済みます。

さらに重要なのは、**データの追加や削除を行っても、高速に「山」を整え直せる**点です。

- **追加・削除の再構築**：計算量は $O(n \log n)$ です。
- **配列との相性**：完全二分木なので、ポインタを使わなくても「普通の配列」で親子関係を計算できるため、メモリ効率が非常に良いです。

![note]

親子関係の計算式 (インデックスが0始まりの場合) は 配列のインデックス (添字) を i とすると、以下の数式で親子を特定できます。

左の子のインデックス: $i \times 2 + 1$

右の子のインデックス: $i \times 2 + 2$

親のインデックス: $(i - 1) / 2$

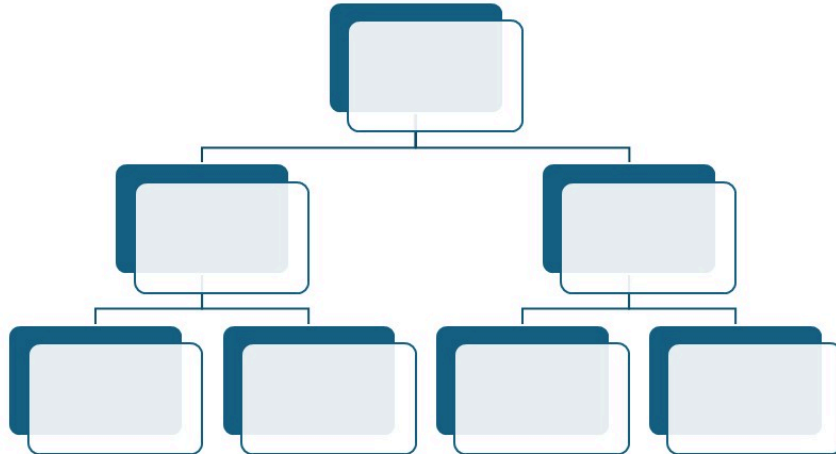
3. ヒープの構築

例として、[5, 2, 7, 3, 6, 1, 4] というバラバラに並んだ配列をツリーに追加して、「親が子より大きい」というルールにしたがって、ヒープを構築します。

Array

0	1	2	3	4	5	6
5	2	7	3	6	1	4

Heap



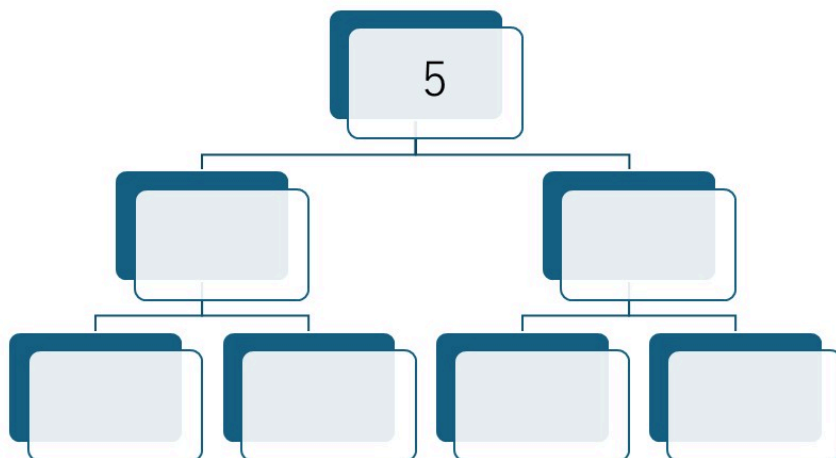
- 5 をツリーに追加

Array

0	1	2	3	4	5	6
	2	7	3	6	1	4

Heap

降順になるように
ヒープを構築する



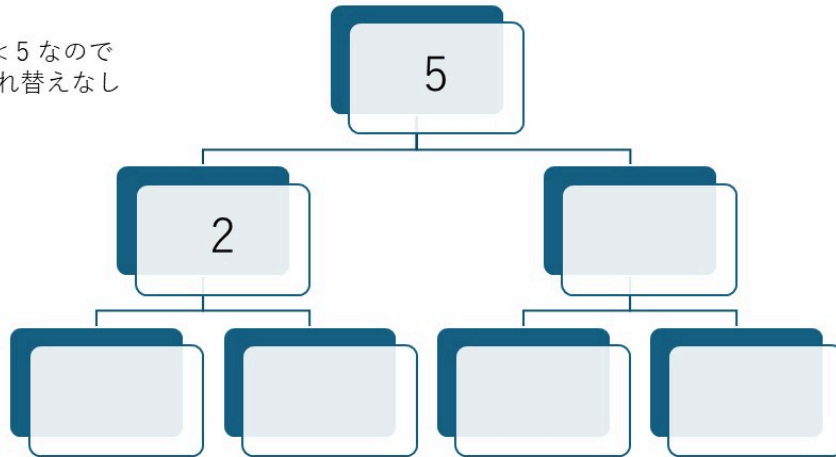
- 2 を追加 → $2 < 5$ なので、**安定**

Array

0	1	2	3	4	5	6
		7	3	6	1	4

Heap

$2 < 5$ なので
入れ替えなし



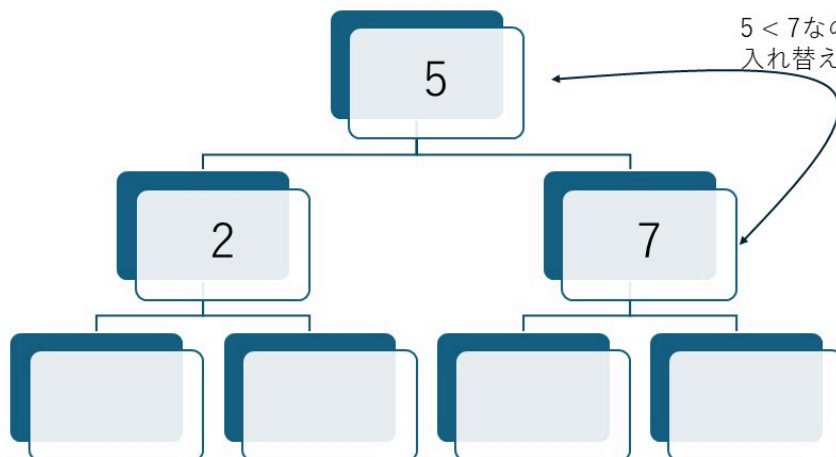
- 7 を追加 → $5 < 7$ なので、**入れ替え**

Array

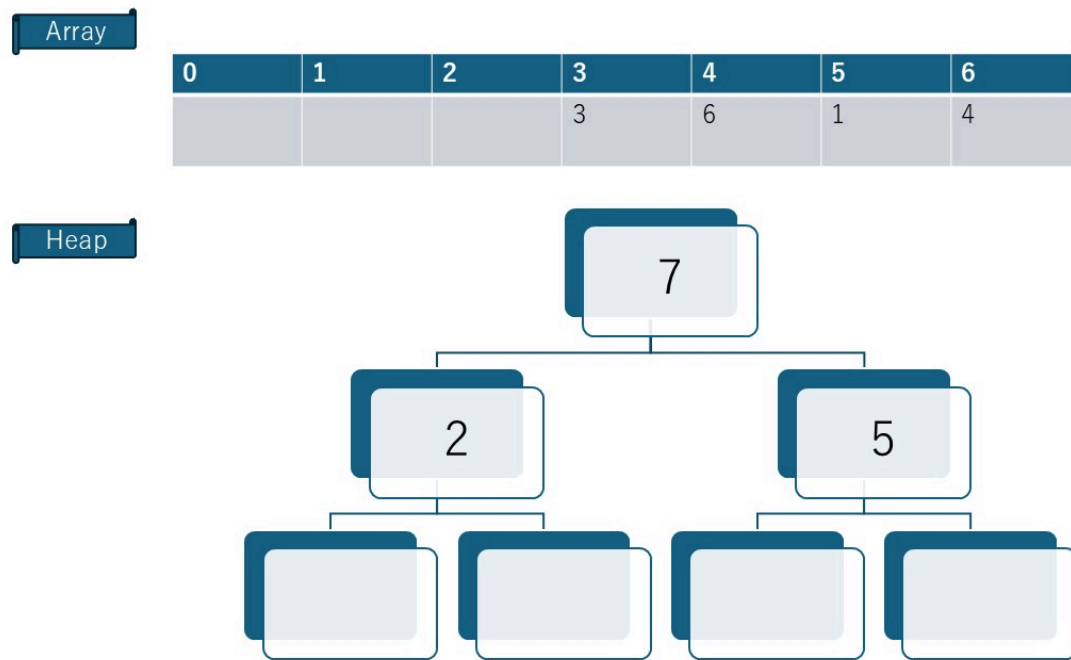
0	1	2	3	4	5	6
			3	6	1	4

Heap

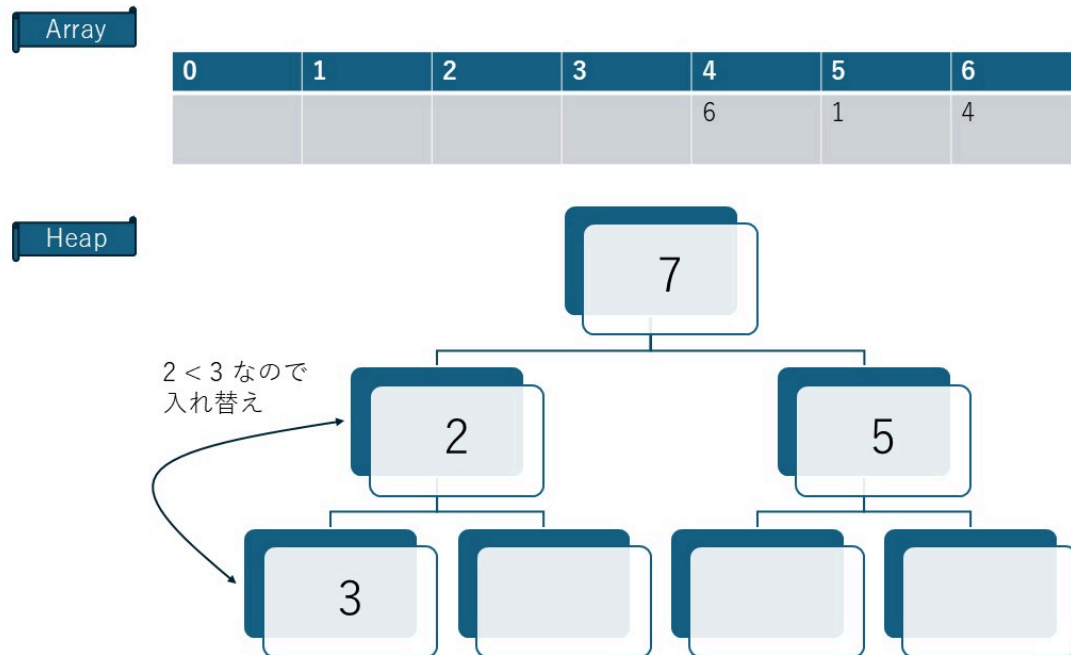
$5 < 7$ なので
入れ替え



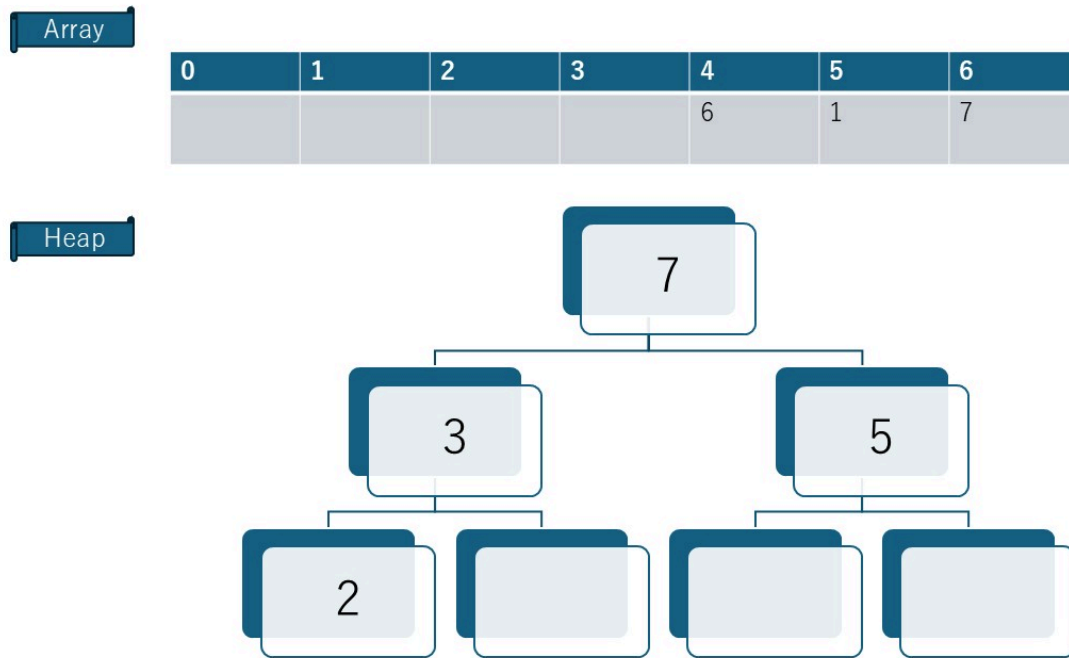
- 入れ替え後、親 > 子のルールに従っているので、**安定**



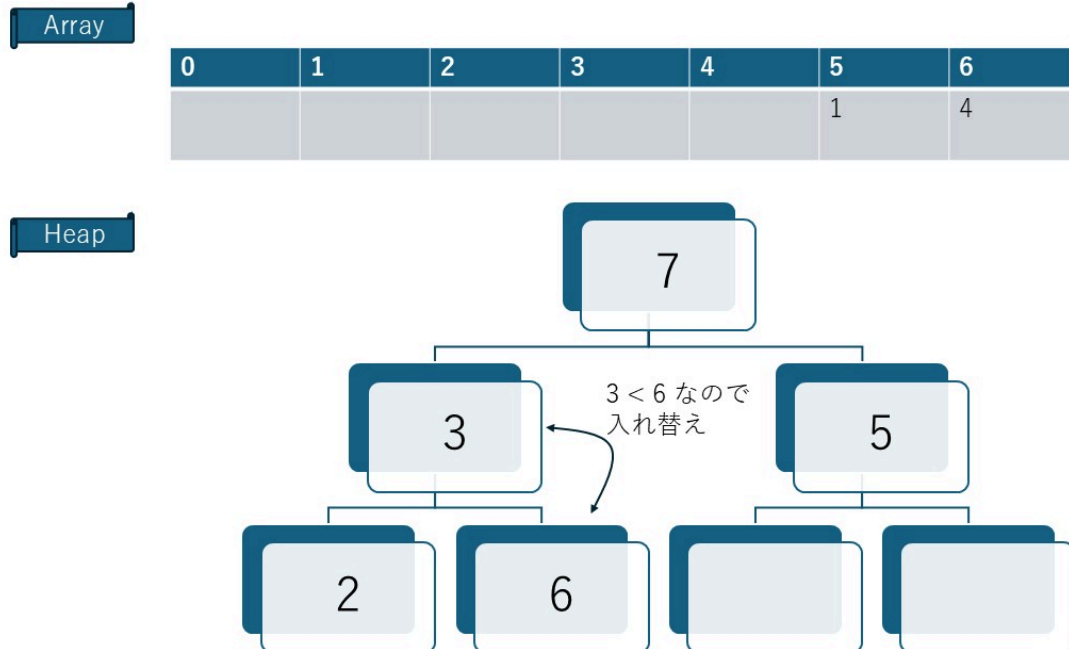
- 3 を追加 → $2 < 3$ なので、**入れ替え**



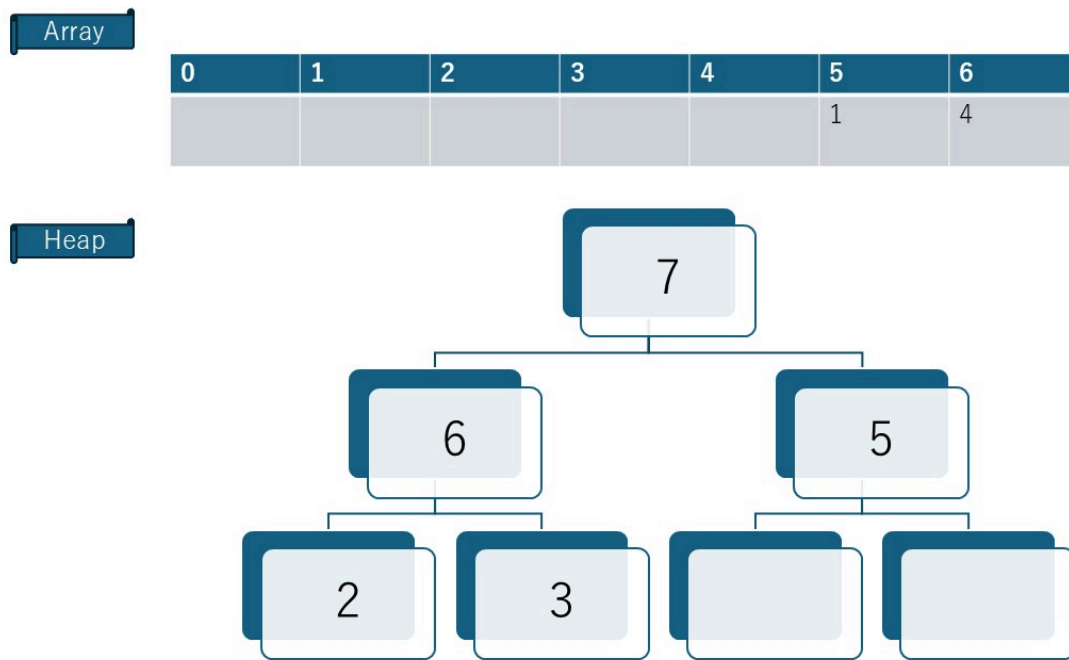
- 入れ替え後、親 > 子のルールに従っているので、**安定**



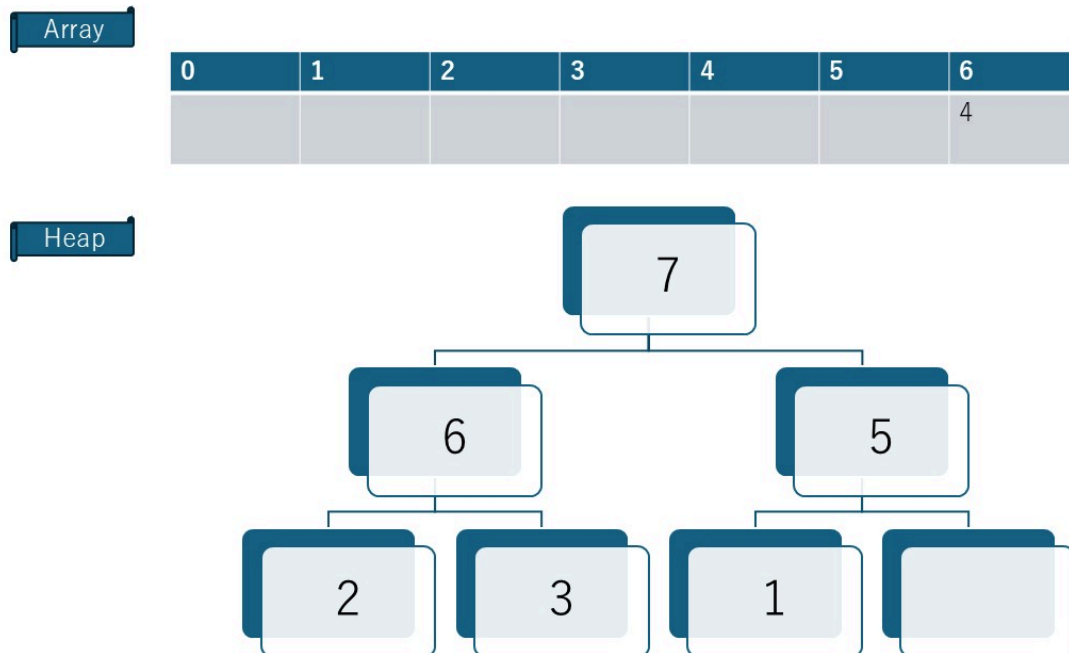
- 6 を追加 → $3 < 6$ なので、**入れ替え**



- 入れ替え後、親 > 子のルールに従っているので、**安定**



- 1 を追加 → $1 < 5$ なので、**安定**

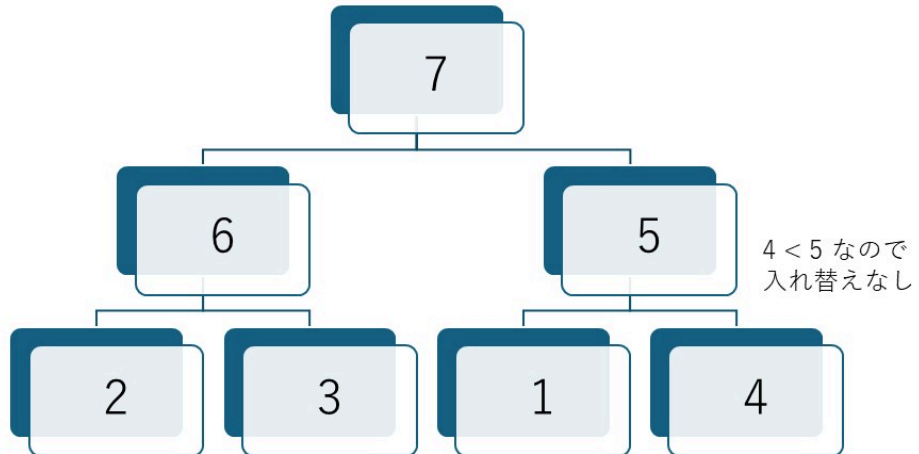


- 4 を追加 → $4 < 5$ なので、安定

Array

0	1	2	3	4	5	6

Heap



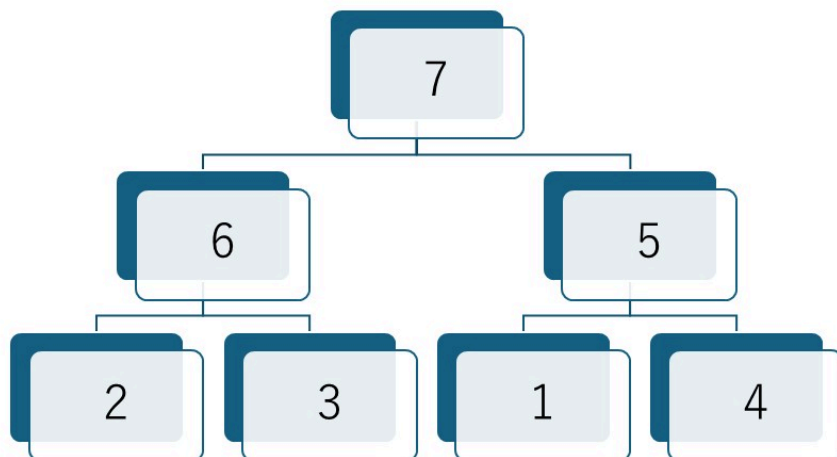
- これで、ヒープ完成!

Array

0	1	2	3	4	5	6

Heap

ヒープ完成



4. ヒープの再構築：ヒープ化 (Heapify)

データを削除（頂点を取り出す）したとき、ヒープは以下の手順で形を整えます。

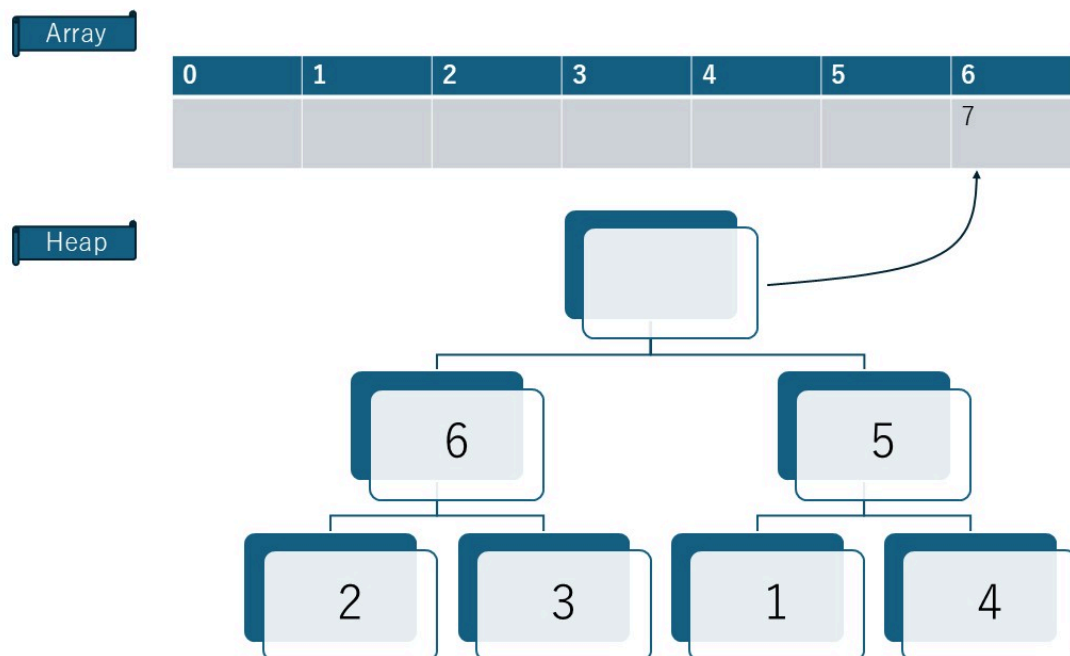
1. 空いた頂点に、一番末尾（右下）の要素を持ってくる。
2. その要素を子と比較し、ルール違反（子が自分より大きい等）なら入れ替える。
3. これを適切な位置に収まるまで繰り返す（沈み込み）。

これを使って、配列をソートすることができます（ヒープソート）。

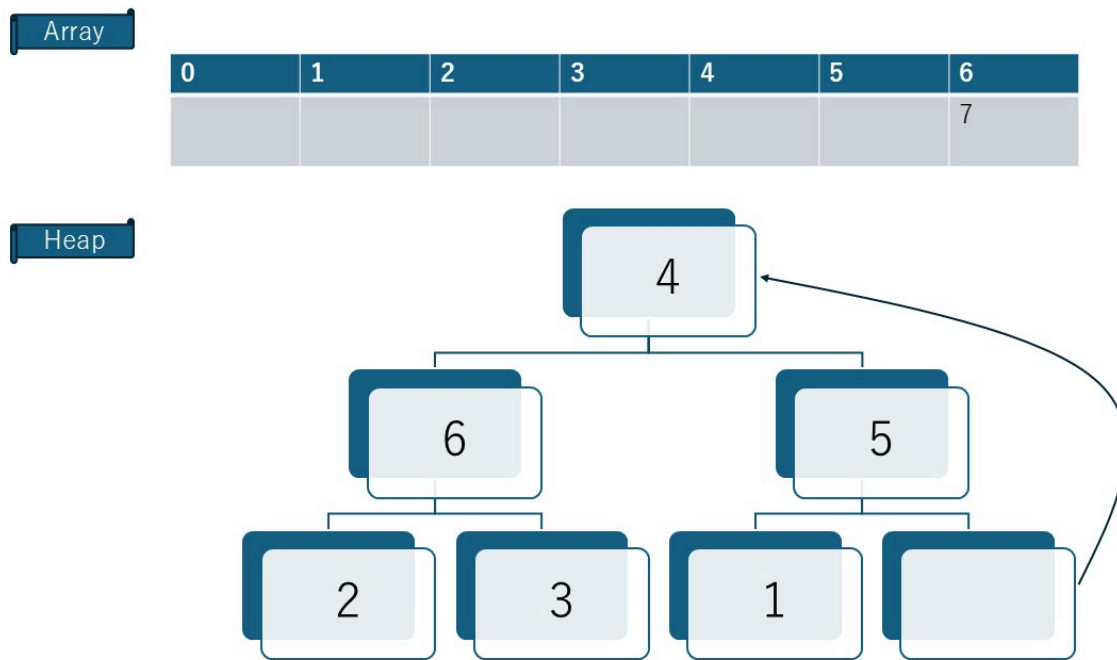
![note]

ヒープソートの実装については、「ヒープソート」の章でくわしく説明します。ここでは、「ヒープの再構築」の例として図解します。

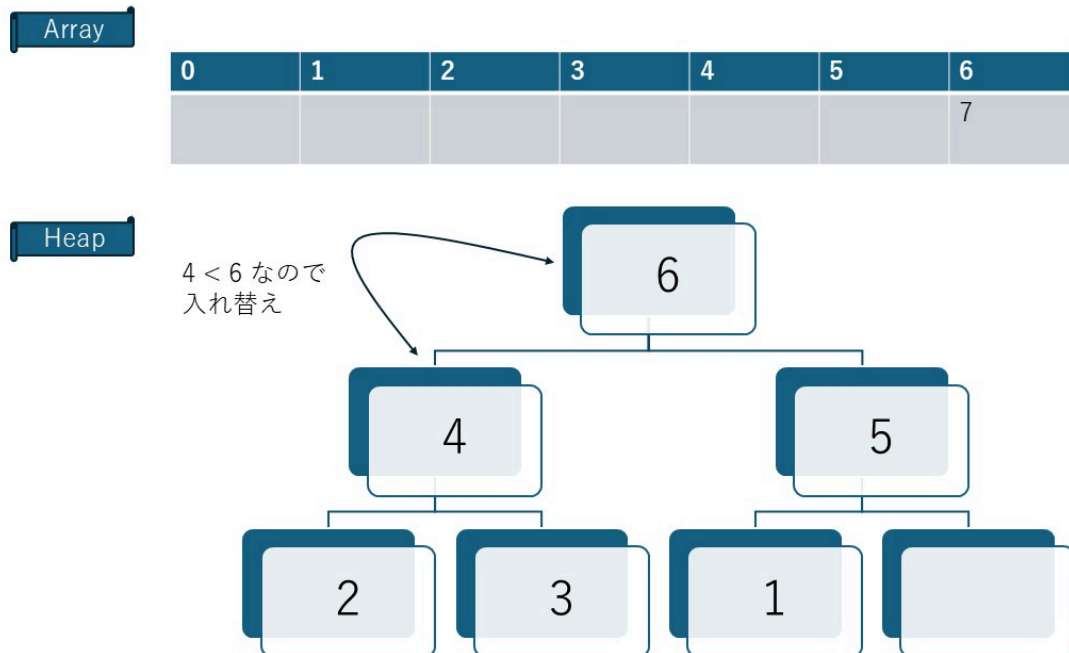
- 頂点の 7 を取り出す



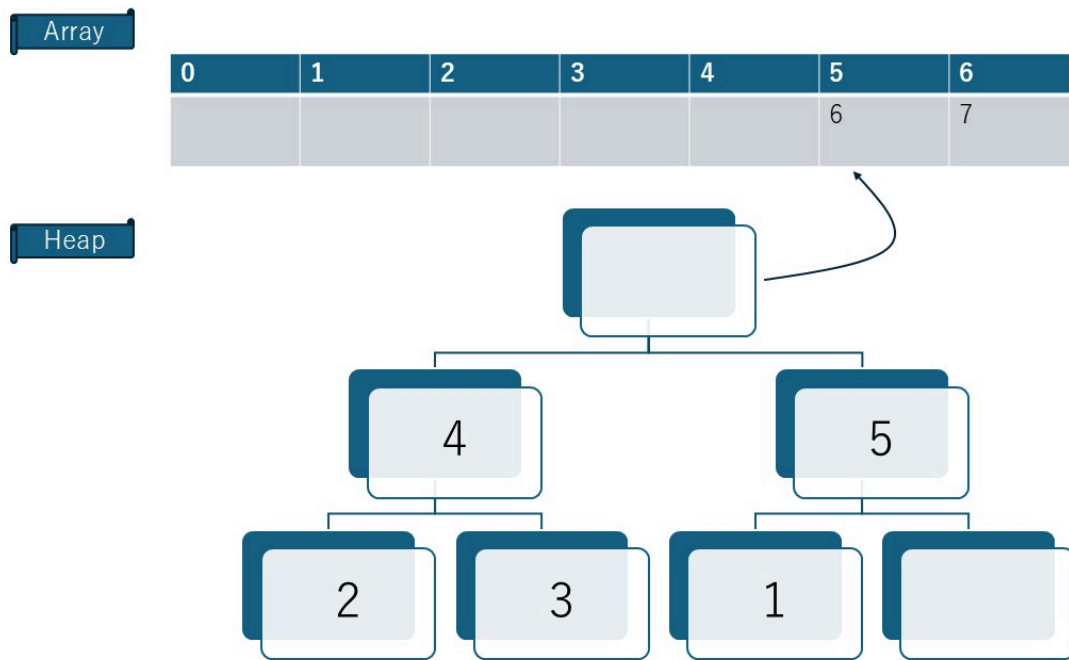
- 右下の 4 を空いた頂点に移動



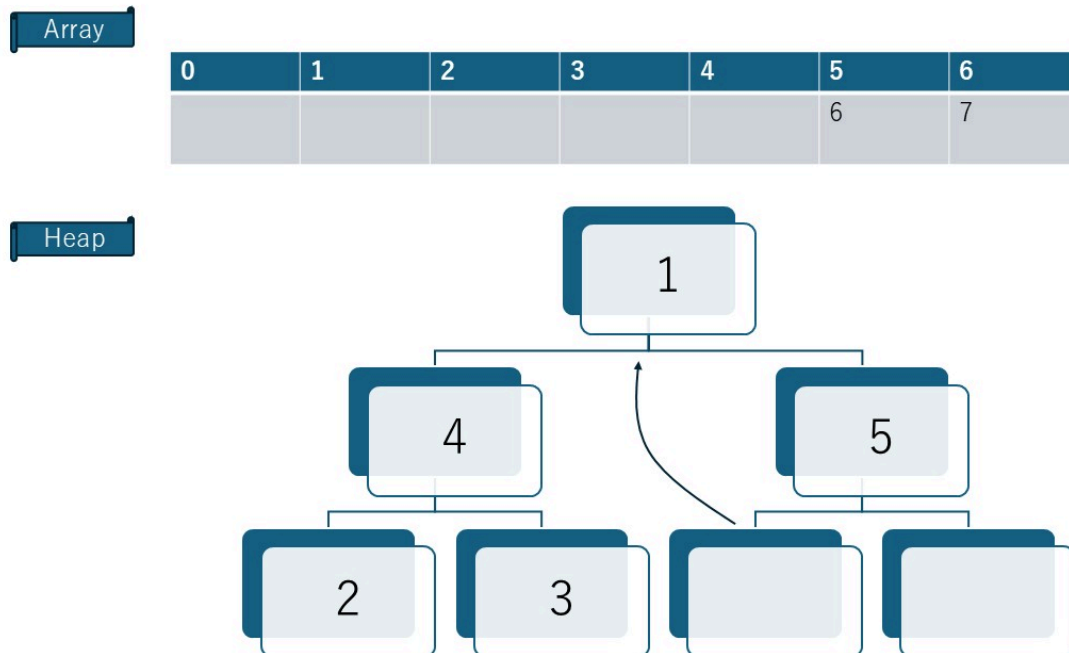
- $4 < 6$ なので、入れ替え



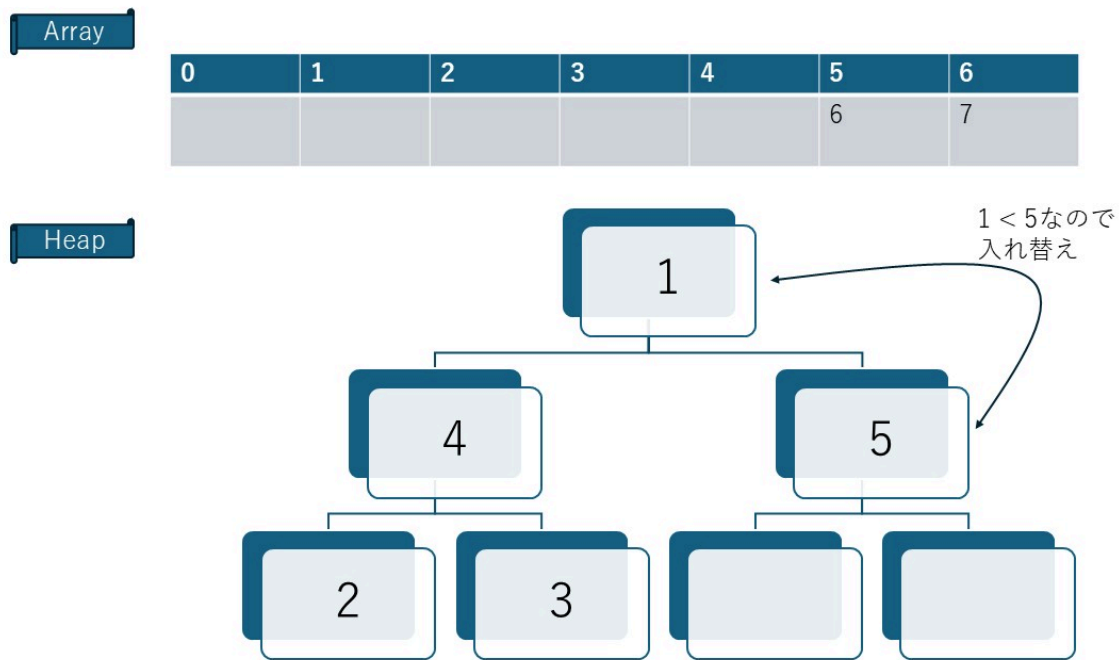
- 頂点の 6 を配列に戻す



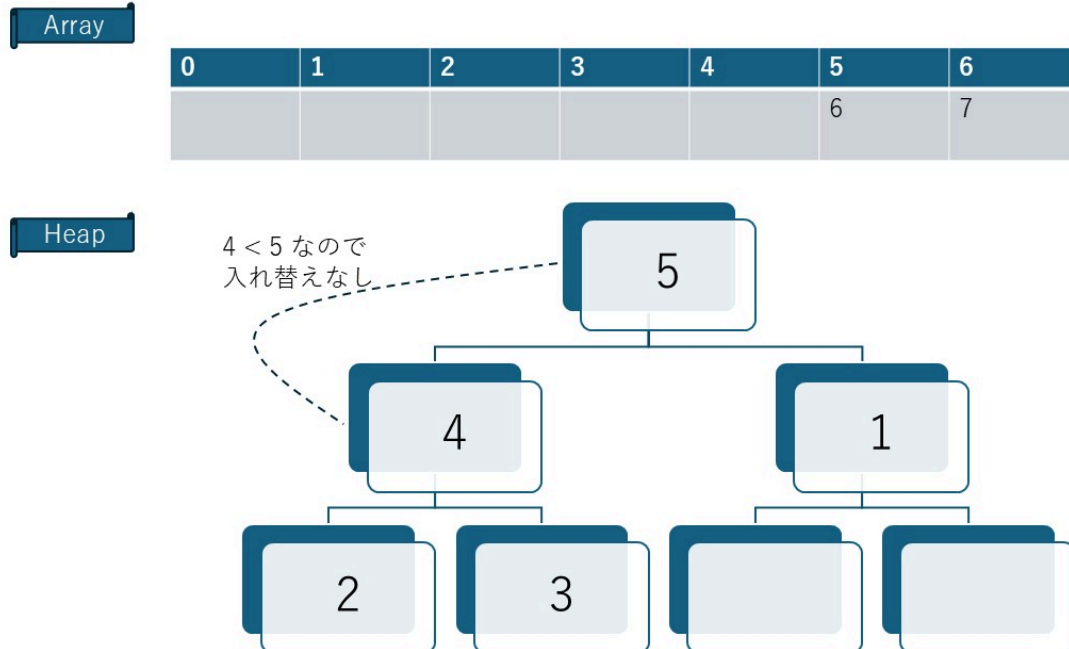
- 右下の 1 を空いた頂点に移動



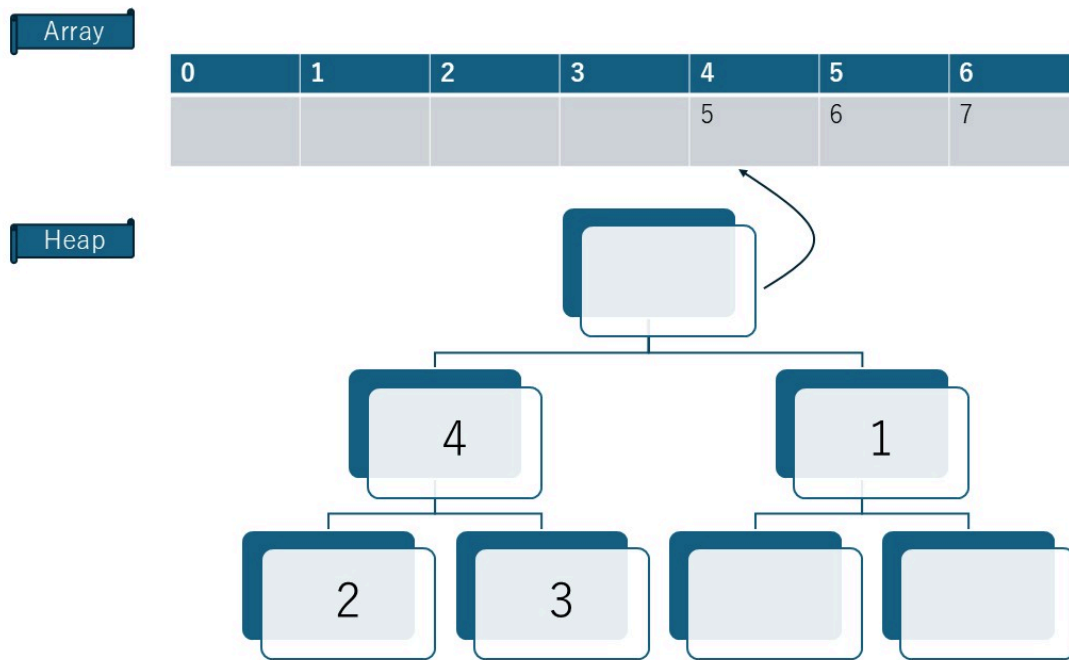
- $1 < 5$ なので、入れ替え



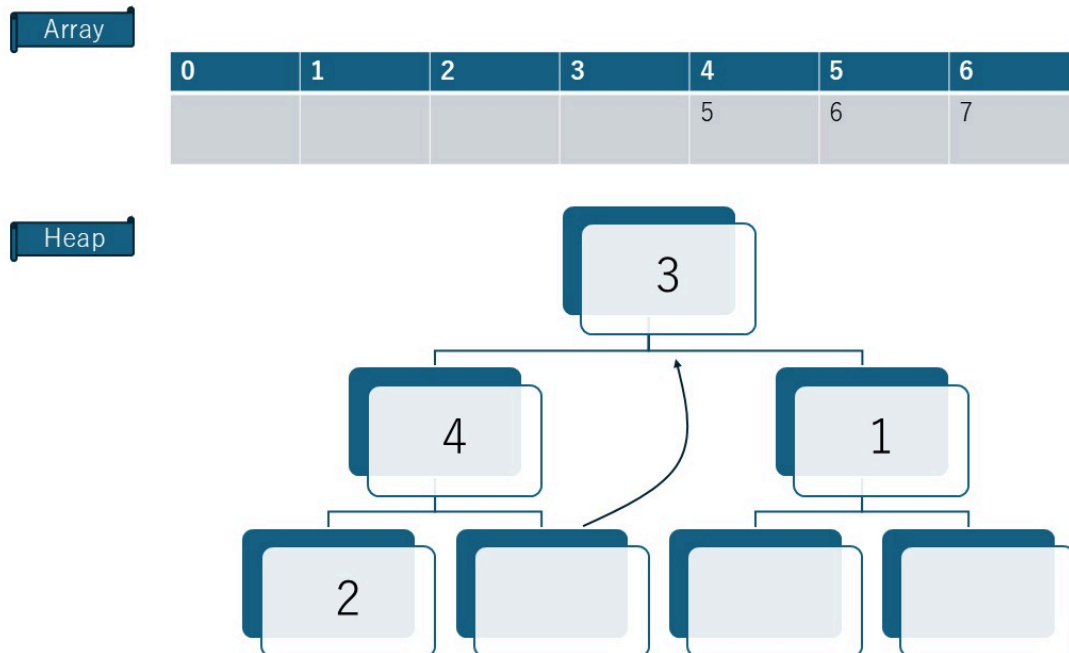
- $4 < 5$ なので、安定



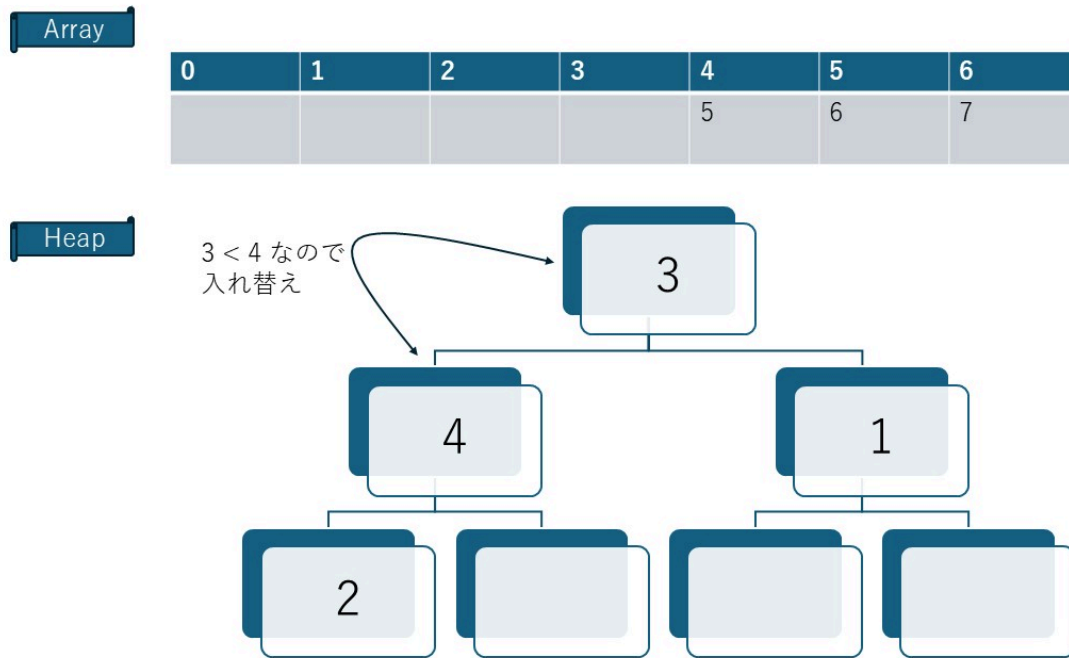
- 頂点の 5 を配列に戻す



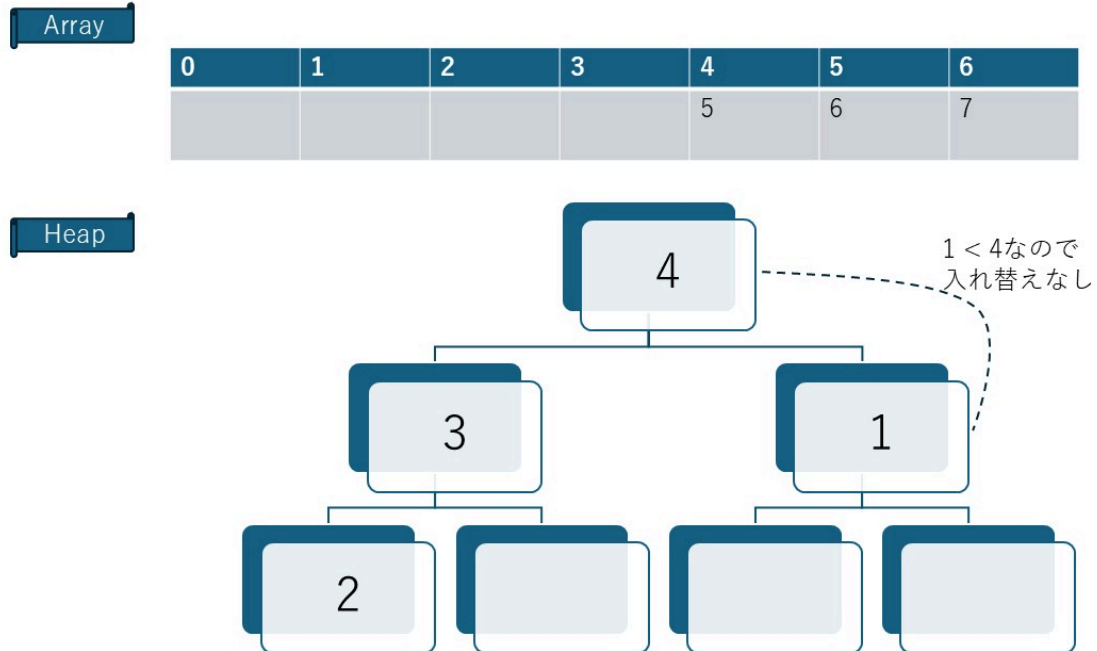
- 右下の 3 を空いた頂点に移動



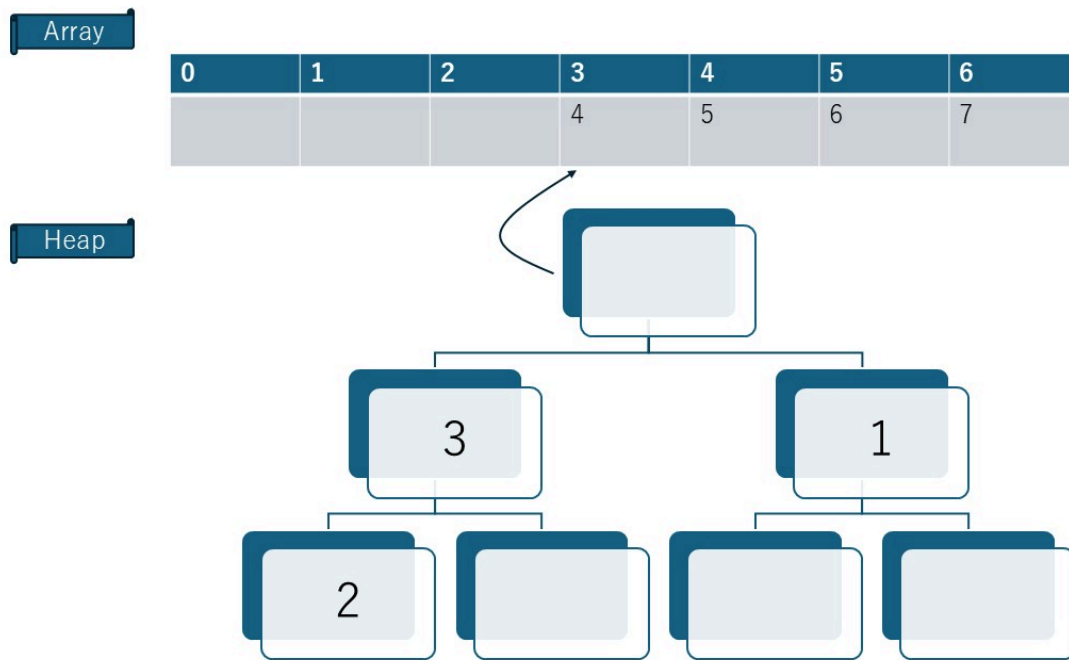
- 3 < 4 なので、入れ替え



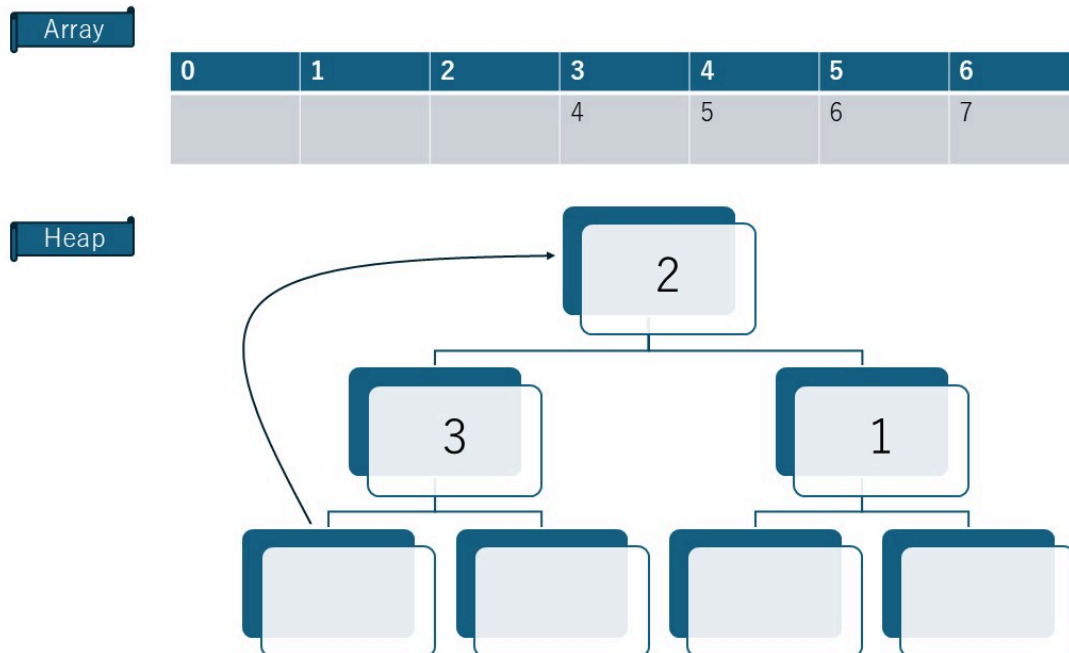
- 1 < 4 なので、安定



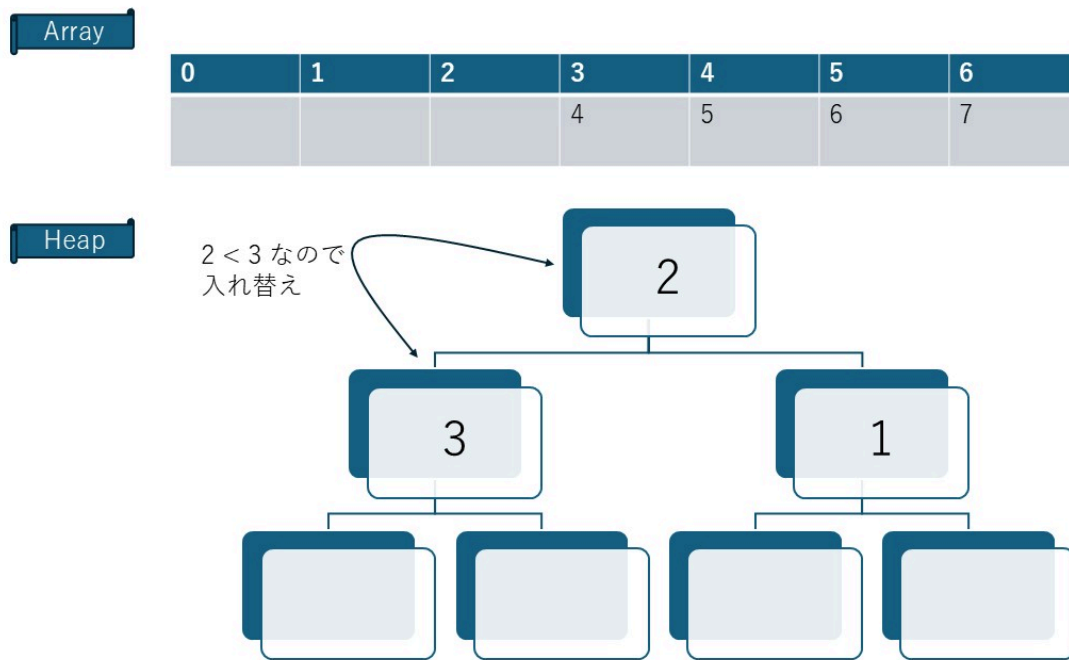
- 頂点の 4 を配列に戻す



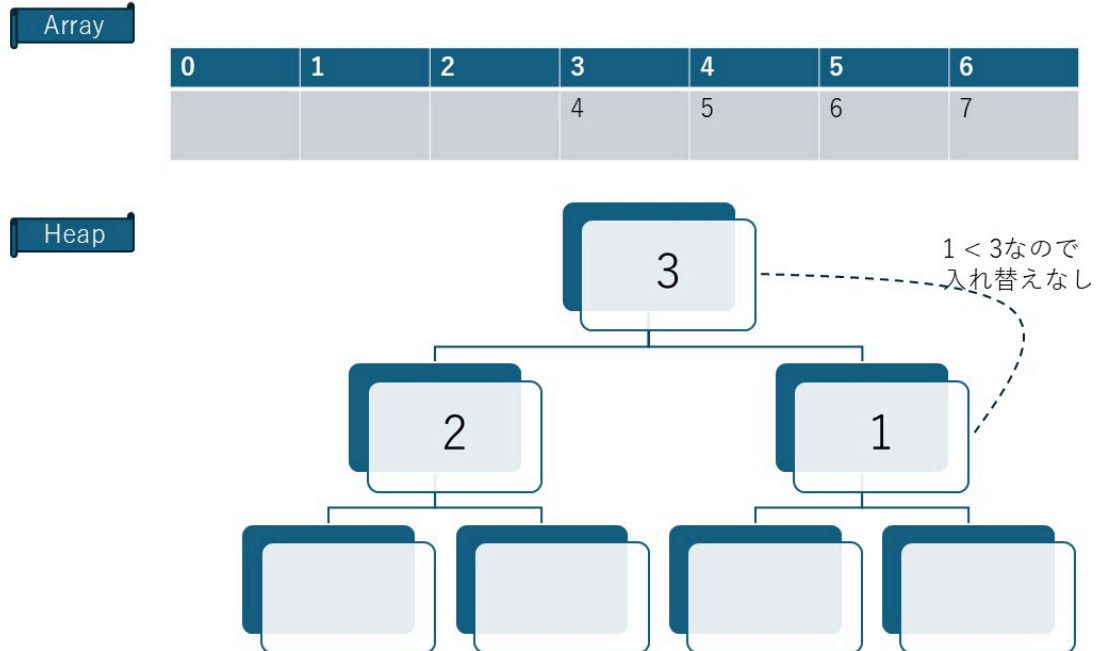
- 右下の 2 を空いた頂点に移動



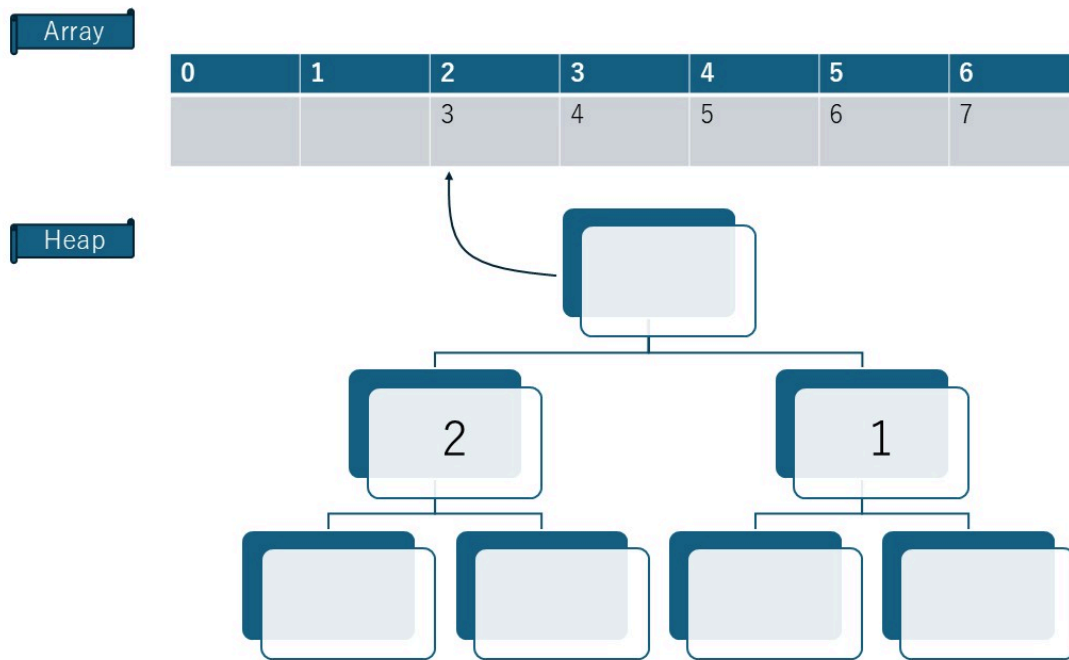
- $2 < 3$ なので、入れ替え



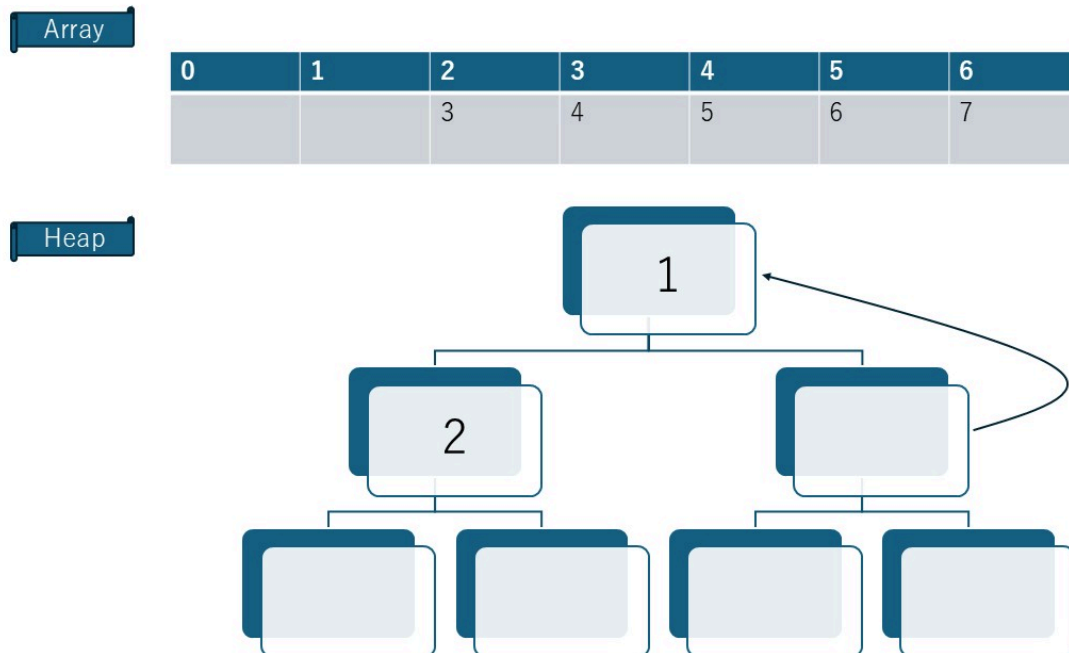
- $1 < 3$ なので、安定



- 頂点の **3** を配列に戻す



- 右下の **1** を空いた頂点に移動



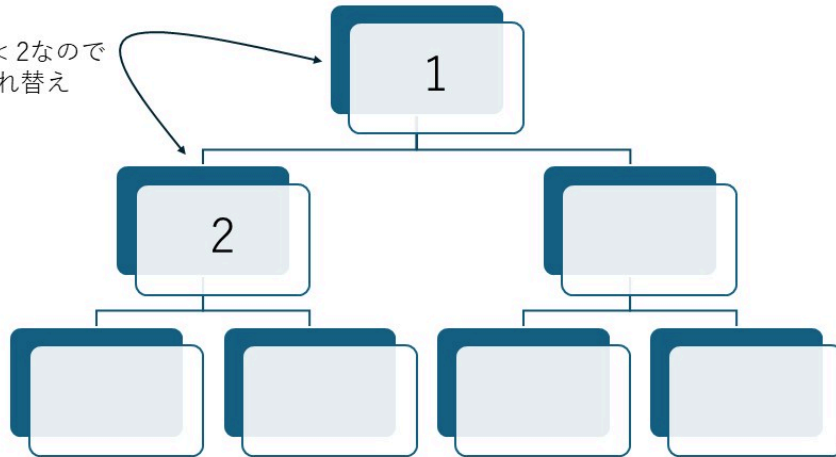
- $1 < 2$ なので、入れ替え

Array

0	1	2	3	4	5	6
		3	4	5	6	7

Heap

$1 < 2$ なので
入れ替え

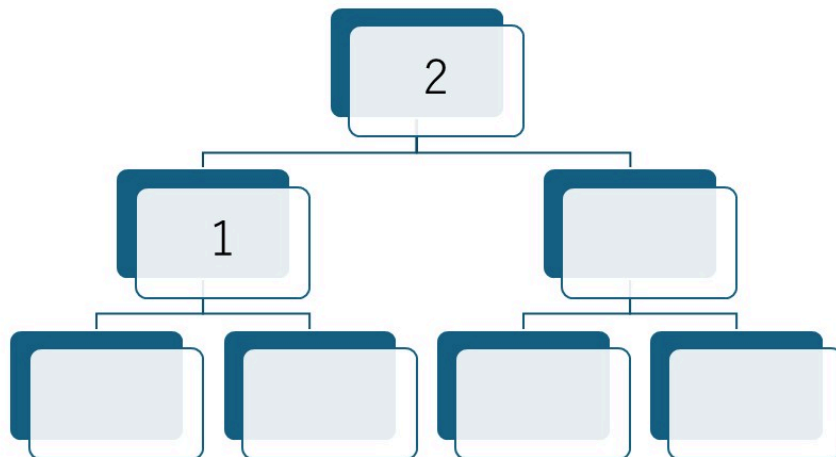


- 安定

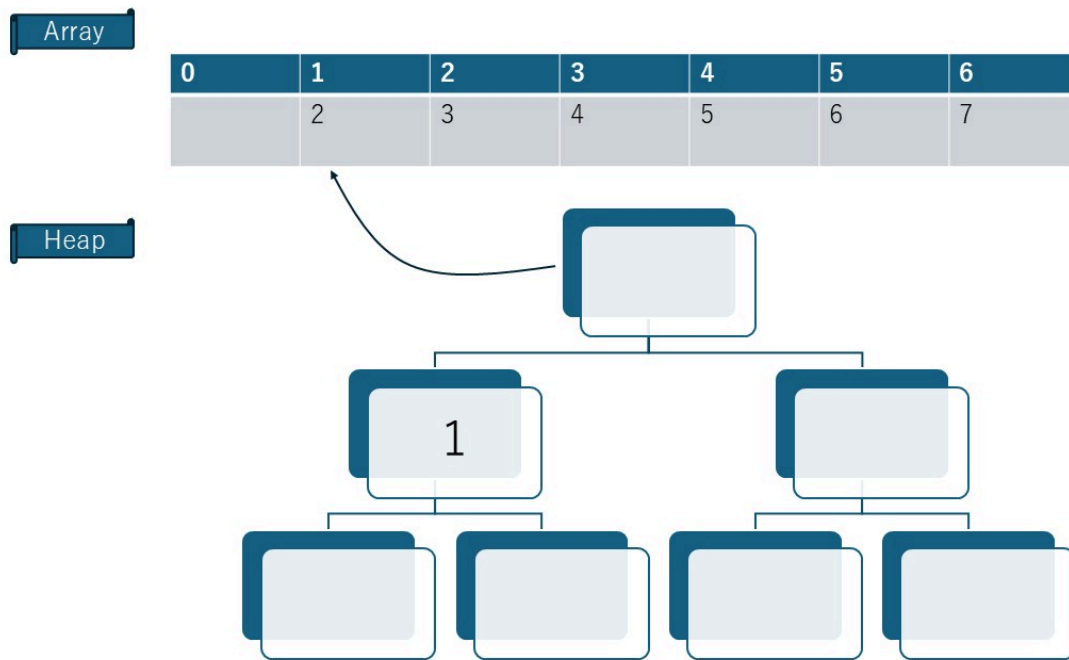
Array

0	1	2	3	4	5	6
		3	4	5	6	7

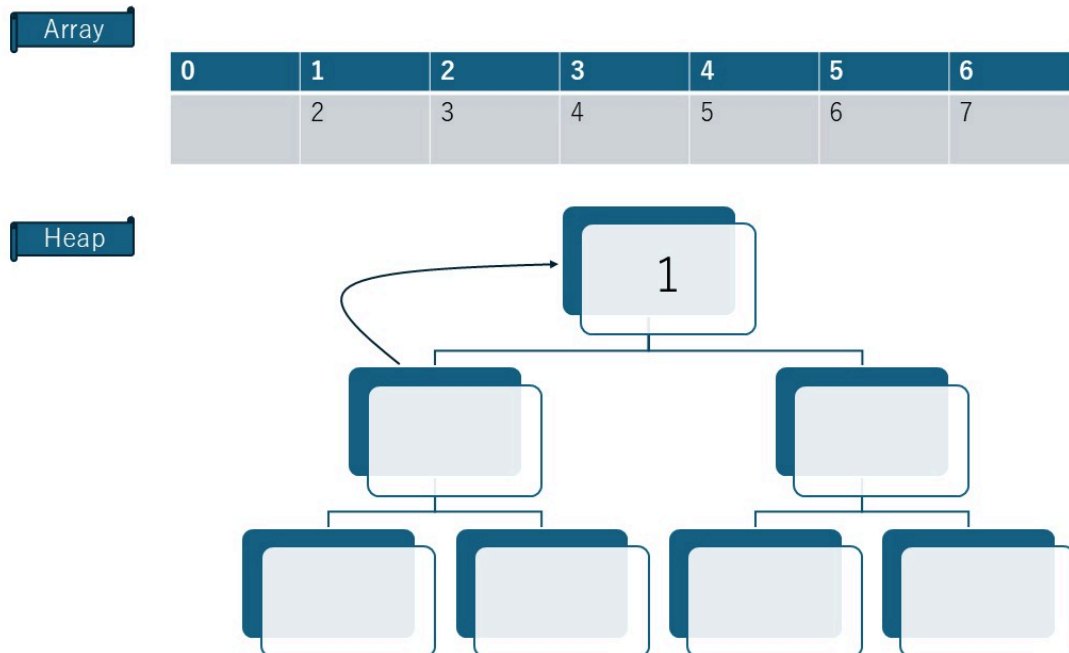
Heap



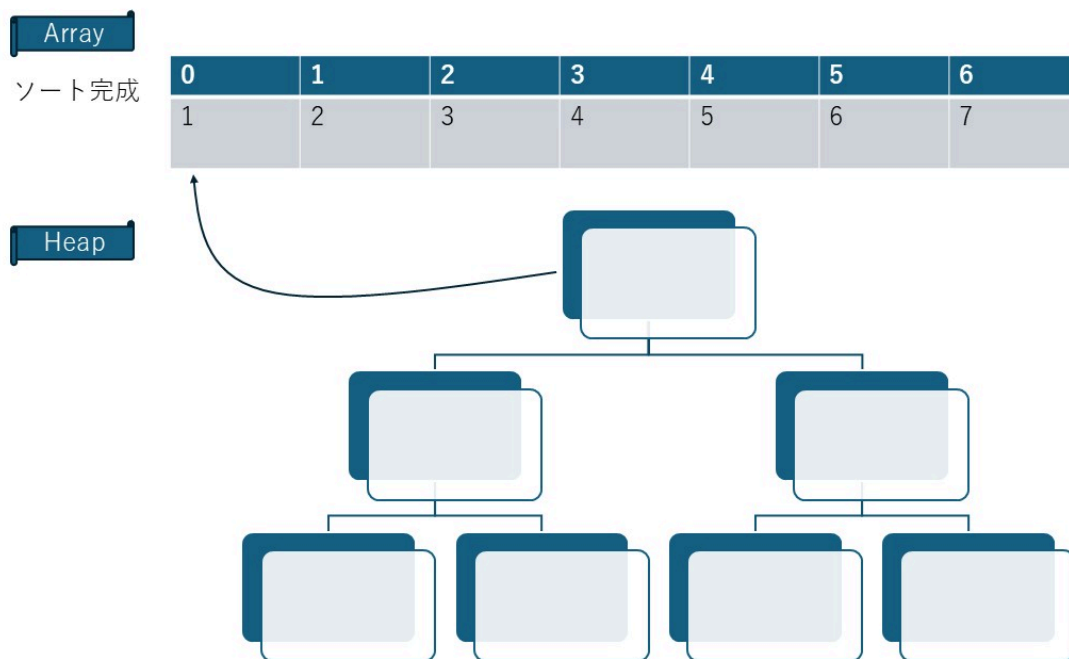
- 頂点の 2 を配列にも戻す



- 右下の 1 を空いた頂点に移動



- 頂点の 1 を配列に戻して、完成



4. 実社会・ITでの活用例

- **優先度付きキュー (Priority Queue) :** 単なる待ち行列（キュー）ではなく、「緊急度の高い人から順に処理する」という仕組みに最適です。
 - **ヒープソート :** ヒープの頂点（最大値）を順に取り出し続けることで整列させるソート法です。
 - **OSのタスクスケジューリング :** CPUが実行すべきタスクの中から、優先順位が最も高いものを瞬時に見つけ出すために使われます。
-